Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Arquitectura del Computador

Simulador de Caché de Alta Fidelidad y Comparación de Políticas de Reemplazo: LRU, FIFO y LFU

Diseño e Implementación en C++17 bajo Complejidad $O(1)$, Clasificación Formal de Fallos según las 3C de Mark D. Hill y Reutilización de Librerías Propias

PROYECTO N° 4:
Çomparación de Políticas de Reemplazo (LRU, FIFO, LFU): Desarrollar un simulador de memoria caché que permita probar diferentes políticas de reemplazo y comparar su eficiencia. Medir el tiempo de ejecución y el hit ratio. Ajustar estructuras para gestionar eficientemente la política seleccionada."

Integrantes del Equipo:

- **Jesús Escorcha** C.I.: 31.107.404
- **Tulio Cordero** C.I.: 30.571.603
- **Samuel Archila** C.I.: 31.505.277

Septiembre de 2026

Resumen

El presente informe técnico expone el diseño, desarrollo, verificación y contrastación experimental de un simulador de memoria caché de alta fidelidad implementado en **C++17** para entornos GNU/Linux, desarrollado en el marco del Proyecto 4 de la cátedra Arquitectura del Computador. El simulador permite evaluar y caracterizar de manera exhaustiva el comportamiento microarquitectónico de tres políticas de reemplazo clásicas: **LRU** (*Least Recently Used*), **FIFO** (*First-In, First-Out*) y **LFU** (*Least Frequently Used*), contrastándolas adicionalmente frente a la política óptima fuera de línea de Belady (**MIN/OPT**) y una política estocástica (**Random**) como línea base.

Para superar las deficiencias algorítmicas de simuladores académicos elementales —cuyas operaciones de selección de víctima o actualización de recencia suelen incurrir en costo lineal $O(N)$—, se diseñaron estructuras de datos compuestas que garantizan una complejidad temporal estrictamente constante $O(1)$ para todas las operaciones críticas. En particular, para LFU se implementó el algoritmo bidimensional de cubetas de Ketan Shah, resolviendo de forma determinista y en tiempo constante los empates de frecuencia por antigüedad LRU. Asimismo, se integraron y reutilizaron de manera directa las estructuras de datos propias desarrolladas durante la carrera (`cola.hpp` y `pila.hpp`), empleando la cola para el control FIFO y la pila como lista de vías libres (*Free List*) de hardware dentro de cada conjunto asociativo, eliminando por completo la sobrecarga de asignación dinámica en tiempo de simulación.

La investigación trasciende la mera medición del *Hit Ratio* al incorporar el cálculo formal del **AMAT** (*Average Memory Access Time*) y los ciclos de parada de procesador (*CPU Stalls*), integrando mediciones de tiempo monotónicas de alta resolución (`std::chrono::steady_clock`), barreras de optimización del compilador mediante ensamblador en línea (`asm volatile`), y un clasificador formal de fallos basado en el **Modelo de las 3C de Mark D. Hill** mediante una caché sombra totalmente asociativa ejecutada en paralelo. La experimentación abarcó 10 conjuntos de datos (sintéticos, estadísticos bajo distribuciones Uniforme, Gaussiana y Zipf, y trazas reales con 100.000 referencias cada una), logrando validar experimentalmente la célebre **Anomalía de Belady** en FIFO y caracterizar la vulnerabilidad de *inercia de frecuencia* en LFU frente a la adaptabilidad temporal de LRU. Finalmente, se presenta una evaluación crítica de los recursos pedagógicos del curso y un prompt estructurado para la generación de infografías mejoradas.

Palabras clave: Memoria Caché, Políticas de Reemplazo, LRU, FIFO, LFU, Complejidad $O(1)$, Modelo 3C de Hill, AMAT, Anomalía de Belady, C++17, Arquitectura del Computador.

Índice

1. El Problema **2**
- 1.1. El Cuello de Botella de von Neumann y el *Memory Wall* . . . 2
- 1.2. El Principio de Localidad de Referencia . . . 2
- 1.3. Microarquitectura de la Caché y Descomposición de Direcciones . . . 2
- 1.4. El Dilema del Reemplazo y la Selección del Bloque Víctima . . . 3
- 1.5. Políticas de Reemplazo Evaluadas . . . 3
- 1.6. Patologías Teóricas de Reemplazo . . . 4
- 1.7. Estrategias de Escritura y Coherencia de Datos . . . 4

2. Técnicas Computacionales para su Solución **5**
- 2.1. Arquitectura del Software: Patrón Strategy y Cero Variables Globales . . . 5

2.2. Estructuras de Datos y Complejidad Temporal Estricta $O(1)$ 5
2.2.1. 1. LRU en $O(1)$ (Lista Doblemente Enlazada + Mapeo de Iteradores) 6
2.2.2. 2. FIFO en $O(1)$ (Cola Secuencial Cronológica) 6
2.2.3. 3. LFU en $O(1)$ (Algoritmo de Cubetas Bidimensionales de Ketan Shah) . . . 6
2.3. Reutilización e Integración de Librerías Propias (`librerias-lpc`) 7
2.4. Clasificación Formal de Fallos: Modelo de las 3C de Mark D. Hill 8
2.5. Modelado de Latencia Real y Paradas de CPU (AMAT y Stalls) 9

3. Análisis y Comparación de Resultados **10**
3.1. Entorno Experimental y Catálogo de Cargas de Trabajo 10
3.2. Tabla Consolidada de Resultados Experimentales 11
3.3. Discusión y Hallazgos de Arquitectura 11
3.4. Comprobación Experimental de la Anomalía de Belady en FIFO 12
3.5. Análisis de Sensibilidad: Barrido de Asociatividad por Vías 12

4. Validación y Propuesta de Mejora al Recurso de Ayuda **13**
4.1. Diagnóstico Crítico del Recurso Actual 13
4.2. Prompt Estructurado Asociado a la Infografía Mejorada 13

5. Conclusiones y Recomendaciones **15**

A. Guía de Compilación, Ejecución y Reproducibilidad **17**

1. El Problema

1.1. El Cuello de Botella de von Neumann y el *Memory Wall*

En la arquitectura de computadores contemporánea, el rendimiento de los sistemas se encuentra condicionado por la disparidad exponencial de velocidad entre las unidades centrales de procesamiento (CPU) y la memoria principal (DRAM), fenómeno ampliamente documentado en la literatura especializada como la brecha de rendimiento procesador-memoria o ***Memory Wall*** [1].

Mientras los procesadores modernos operan a frecuencias de gigahercios ejecutando múltiples instrucciones por ciclo de reloj (con tiempos de ciclo inferiores a 0,5 ns), la memoria principal de acceso aleatorio dinámica (DRAM) requiere típicamente entre 50 ns y 100 ns para satisfacer una solicitud de acceso (equivalente a un retardo de 100 a 200 ciclos de reloj de procesador). Esta alta latencia proviene de la naturaleza capacitiva de las celdas DRAM, las cuales requieren ciclos de precarga de línea de bits, amplificación de detección y operaciones periódicas de refresco destructivo.

Para amortiguar esta penalización temporal crítica, se interpone entre la CPU y la DRAM una jerarquía de memorias intermedias ultra-rápidas construidas con tecnología de memoria de acceso aleatorio estática (**SRAM**). La memoria SRAM utiliza transistores biestables (típicamente 6 transistores por celda básica), lo cual permite responder a accesos de lectura y escritura en 1 a 3 ciclos de procesador sin necesidad de refresco. Sin embargo, su complejidad de integración física, densidad limitada y elevado consumo energético impiden fabricar memorias principales enteramente en SRAM, limitando su capacidad a tamaños de kilobytes o megabytes (L1, L2, L3).

1.2. El Principio de Localidad de Referencia

La viabilidad económica y técnica de la jerarquía de memoria descansa sobre el **Principio de Localidad de Referencia**, el cual establece que los programas informáticos no acceden a su espacio de memoria de forma uniformemente aleatoria, sino que exhiben dos patrones deterministas fundamentales:

1. **Localidad Temporal:** Si una posición de memoria es referenciada en un instante de tiempo t, existe una probabilidad abrumadoramente alta de que vuelva a ser referenciada en un futuro cercano $t + \Delta t$ (e.g., variables de control de bucles, acumuladores, pilas de llamadas).

2. **Localidad Espacial:** Si una posición de memoria con dirección A es referenciada, existe una alta probabilidad de que las posiciones contiguas $A + 1, A + 2, \ldots$ sean referenciadas en breve (e.g., ejecución secuencial de código máquina, recorrido contiguo de arreglos o matrices).

1.3. Microarquitectura de la Caché y Descomposición de Direcciones

Para aprovechar la localidad espacial, la información entre la memoria principal y la memoria caché se transfiere en bloques atómicos indivisibles denominados **líneas o bloques de caché** (típicamente de 64 bytes).

Cuando la Unidad de Manejo de Memoria (**MMU**) procesa una dirección de memoria física de n bits (e.g., 64 bits), la segmenta en tres campos disjuntos de hardware:

$$\text{Dirección de } n \text{ bits} = [\text{ Tag (Etiqueta) }] \parallel [\text{ Index (Índice de Conjunto) }] \parallel [\text{ Offset (Desplazamiento) }] \tag{1}$$

- **Offset (b bits):** Determina el byte exacto dentro del bloque de tamaño $B = 2^b$ bytes. Se calcula como:

$$b = \log_2(B) \tag{2}$$

- **Index (s bits):** Selecciona el conjunto asociativo (*Set*) específico dentro de la caché. Si la caché posee una capacidad total C bytes dividida en S conjuntos con asociatividad de W vías ($C = S \times W \times B$):

$$S = \frac{C}{W \times B}, \quad s = \log_2(S) \tag{3}$$

- **Tag (t bits):** Campo superior restante que identifica de forma unívoca el bloque de memoria DRAM que reside actualmente en una vía determinada:

$$t = n - s - b \tag{4}$$

Ante una solicitud de acceso a una dirección, la caché realiza la siguiente secuencia de eventos:

1. Decodifica el campo *Index* y activa el conjunto correspondiente.
2. Compara en paralelo el campo *Tag* de la dirección solicitada contra los *Tags* de todas las W vías del conjunto que tengan activo el bit de validez (*Valid Bit*).
3. Si coincide algún Tag válido, se produce un **Cache Hit** (acierto). El dato se extrae en 1,0 ciclo SRAM y se remite al procesador.
4. Si ninguna vía coincide o el conjunto está vacío, se produce un **Cache Miss** (fallo). La CPU experimenta una parada obligada (*CPU Stall*) mientras el bloque es solicitado a través del bus del sistema hacia la DRAM lenta.

1.4. El Dilema del Reemplazo y la Selección del Bloque Víctima

Cuando ocurre un fallo y el conjunto indexado se encuentra completamente saturado —es decir, todas sus W vías están ocupadas por bloques válidos—, la caché se enfrenta al problema crucial del **Reemplazo de Bloques**: se debe expulsar uno de los bloques presentes (denominado **bloque víctima**) para conceder su vía al nuevo bloque entrante.

La decisión de cuál vía desalojar no es trivial: expulsar un bloque que será requerido por el procesador en los ciclos subsiguientes provocará un nuevo fallo casi inmediato, encadenando penalizaciones masivas de DRAM y degradando severamente el caudal de procesamiento (*throughput*) de la CPU. Por ende, la política de reemplazo es el componente algorítmico rector que optimiza el aprovechamiento del hardware de caché.

1.5. Políticas de Reemplazo Evaluadas

El Proyecto 4 exige implementar y contrastar exhaustivamente tres políticas de reemplazo clásicas, las cuales complementamos en este trabajo con dos políticas de referencia:

1. **LRU (*Least Recently Used*):** Desaloja el bloque que no ha sido referenciado durante el lapso de tiempo más prolongado. Se fundamenta sólidamente en el principio de localidad temporal: si un bloque fue usado recientemente, es muy probable que vuelva a ser usado pronto; por ende, el bloque cuya última referencia sea la más lejana en el pasado es el candidato con menor probabilidad de reutilización inmediata.
2. **FIFO (*First-In, First-Out*):** Desaloja el bloque que ingresó primero al conjunto, siguiendo un orden cronológico estricto de llegada. Trata a la memoria caché como una tubería o cola secuencial. Su ventaja estriba en su simplicidad conceptual; sin embargo, ignora la frecuencia y la recencia de uso, lo cual puede provocar el desalojo de bloques de datos altamente activos simplemente porque ingresaron hace mucho tiempo.

3. **LFU (*Least Frequently Used*):** Asocia a cada vía un contador de frecuencia de accesos. Ante un conflicto de reemplazo, desaloja el bloque que registra el menor número de referencias acumuladas. Se orienta a proteger bloques altamente populares o densamente consultados. No obstante, en hardware real presenta dos desafíos críticos: la necesidad de un criterio determinista para desempatar bloques con idéntica frecuencia mínima, y la patología de *inercia de frecuencia* (un bloque referenciado intensamente en el pasado retiene un contador elevado y nunca es expulsado, aun cuando ya no vuelva a ser utilizado).

4. **MIN / OPT (Óptimo Teórico de Belady):** Desaloja el bloque cuyo próximo acceso en la secuencia de ejecución se encuentre más lejano en el tiempo futuro. Demostrado por L. A. Belady en 1966 [3], esta política es provadamente óptima (garantiza el mínimo absoluto de fallos posible para cualquier tamaño de caché). Como requiere clarividencia del futuro, es físicamente irrealizable en hardware en línea, pero funge en este proyecto como el límite superior teórico (*ground truth*) para evaluar la eficiencia relativa de LRU, FIFO y LFU.

5. **Random (Línea Base Estocástica):** Selecciona la víctima de forma pseudoaleatoria uniforme entre las W vías del conjunto. Carece de estado de seguimiento de uso, sirviendo como línea base inferior para cuantificar la ganancia neta aportada por la inteligencia algorítmica de LRU, FIFO y LFU.

1.6. Patologías Teóricas de Reemplazo

La investigación de políticas de reemplazo requiere analizar fenómenos patológicos formales:

- **Anomalía de Belady:** Propiedad contra-intuitiva descubierta en algoritmos no pertenecientes a la familia de *Stack Algorithms* (Algoritmos de Pila), en los cuales **un incremento en la capacidad física o número de vías de la caché produce paradójicamente un aumento en la cantidad total de fallos**. FIFO es la política arquetípica susceptible a esta patología, mientras que LRU y OPT son algoritmos de pila demostrablemente inmunes a ella.

- **Bucle de Thrashing (*Trashing por Bucle*):** Ocurre cuando un bucle de programa itera secuencialmente sobre un conjunto de bloques de tamaño $K = W + 1$. Al ser $K > W$, cada nueva lectura desaloja precisamente el bloque que se requerirá en el ciclo inmediato siguiente, provocando un colapso catastrófico al 0 % de tasa de aciertos tanto en LRU como en FIFO.

1.7. Estrategias de Escritura y Coherencia de Datos

El diseño de caché debe contemplar la coherencia de los datos ante operaciones de escritura (`STORE`):

- **Write-Through:** Toda escritura actualiza de forma simultánea la vía en la caché y el bloque correspondiente en la DRAM. Garantiza coherencia inmediata, pero satura el bus del sistema y somete al procesador a penalizaciones constantes de DRAM en cada escritura.

- **Write-Back con Bit de Suciedad (*Dirty Bit*):** Las escrituras modifican exclusivamente la vía en caché SRAM en 1 ciclo, marcando el bloque con una bandera lógica de modificación (*Dirty* = 1). La escritura hacia la DRAM se difiere hasta el instante en que el bloque sucio es elegido como víctima para ser expulsado. Nuestro simulador modela fielmente esta estrategia, contabilizando los *Write-backs* diferidos emitidos al expulsar bloques modificados.

2. Técnicas Computacionales para su Solución

2.1. Arquitectura del Software: Patrón Strategy y Cero Variables Globales

Para garantizar una separación rigurosa de responsabilidades, modularidad y extensibilidad, el software fue diseñado bajo el patrón arquitectónico **Strategy** del Gang of Four (GoF) [5].

En lugar de incrustar lógica condicional mediante sentencias `switch-case` dentro del controlador de caché, se definió una interfaz abstracta pura denominada `PoliticaReemplazo`. La clase `Cache` interactúa con sus políticas exclusivamente mediante polimorfismo dinámico, desconociendo por completo las estructuras de datos internas que soportan a cada algoritmo.

```
namespace cachesim {
    struct ContextoAcceso {
        DireccionMemoria bloque;
        std::size_t indiceTraza;
    };

    class PoliticaReemplazo {
    public:
        virtual ~PoliticaReemplazo() = default;
        virtual std::string getNombre() const = 0;
        virtual void registrarAcierto(int via, const ContextoAcceso &ctx) = 0;
        virtual void registrarInsercion(int via, const ContextoAcceso &ctx) = 0;
        virtual int seleccionarVictima() = 0;
        virtual std::string describirEstado() const = 0;
    };
}
```

Listing 1: Interfaz abstracta polimórfica `PoliticaReemplazo` en C++17.

Principio de Cero Variables Globales Mutables: Un pilar técnico innegociable de la implementación es la ausencia absoluta de variables globales de estado. Toda la información de configuración (`ConfiguracionCache`), estructura física (`Cache`, `ConjuntoCache`, `LineaCache`), métricas estadísticas (`EstadisticasCache`) y clasificadores sombra viaja encapsulada dentro de instancias independientes. Esto confiere al simulador tres ventajas científicas esenciales:

1. **Reproducibilidad Determinista:** Múltiples simulaciones pueden ejecutarse de forma secuencial o paralela sin que queden residuos en memoria estática.
2. **Aislamiento de Cargas:** Las evaluaciones de barridos de parámetros no sufren contaminación cruzada.
3. ***Thread-Safety*:** El motor se encuentra preparado para simulaciones concurrentes multihilo en entornos de cómputo de alto rendimiento.

2.2. Estructuras de Datos y Complejidad Temporal Estricta $O(1)$

El requerimiento medular del proyecto consiste en garantizar que el simulador opere a escala masiva (procesando millones de referencias en milisegundos). Para ello, cada una de las políticas de reemplazo fue estructurada para garantizar una complejidad computacional estrictamente acotada a $O(1)$ en todas sus operaciones primitivas:

2.2.1. 1. LRU en $O(1)$ (Lista Doblemente Enlazada + Mapeo de Iteradores)

Una implementación ingenua de LRU almacena una marca de tiempo (*timestamp*) en cada vía y, ante un desalojo, busca linealmente el valor mínimo en $O(W)$, donde W es el número de vías. Para una caché de 16 o 32 vías, este costo se torna prohibitivo.

Nuestra solución combina:

- Una lista doblemente enlazada (`std::list<int> ordenRecencia`) que mantiene los identificadores de vía ordenados desde el más recientemente usado (*MRU*, en la cabeza) hasta el menos recientemente usado (*LRU*, en la cola).
- Un arreglo contiguo indexado directamente por número de vía (`std::vector<std::list<int>::iterator> posicionVia`).

Cuando una vía experimenta un acierto (*Hit*), se consulta su iterador en tiempo $O(1)$ y se reubica al frente de la lista mediante la función intrínseca `std::list::splice`:

$$T_{\text{Hit_LRU}} = O(1) \quad \text{(reasignación de punteros prev/next)} \tag{5}$$

Cuando se requiere desalojar una víctima, se extrae el elemento ubicado en la cola de la lista (`ordenRecencia.back()`) en tiempo constante:

$$T_{\text{Victima_LRU}} = O(1) \tag{6}$$

2.2.2. 2. FIFO en $O(1)$ (Cola Secuencial Cronológica)

La política FIFO no altera su ordenamiento interno ante aciertos de caché, preservando rigurosamente el instante en el que cada bloque fue insertado. Se gestiona mediante una cola secuencial pura:

- Inserción: `colaLlegada.encolar(via)` en tiempo $O(1)$.
- Selección de Víctima: Se lee la cabeza de la cola (`colaLlegada.getFirst()`) y se desencola en $O(1)$.

2.2.3. 3. LFU en $O(1)$ (Algoritmo de Cubetas Bidimensionales de Ketan Shah)

El algoritmo tradicional de LFU requiere escanear los contadores de todas las vías en $O(W)$ para hallar el mínimo. Implementar LFU en verdadero tiempo $O(1)$ es uno de los problemas más complejos de estructuras de datos en sistemas de almacenamiento, resuelto formalmente por Ketan Shah et al. en 2010 [4].

Implementamos dicha técnica mediante una estructura entrelazada bidimensional:

1. **Eje Horizontal (Cubetas de Frecuencia):** Una tabla hash (`std::unordered_map<uint64_t, std::list<int» cubetasFrecuencia`) donde la clave es el valor exacto de frecuencia F, y el valor es una lista enlazada con las vías que poseen dicha frecuencia.
2. **Eje Vertical (Desempate LRU):** Dentro de cada lista de frecuencia F, los nodos se ordenan según su recencia de acceso.
3. **Puntero de Frecuencia Mínima (`frecuenciaMinima`):** Registra de manera persistente la menor frecuencia poblada en el sistema.

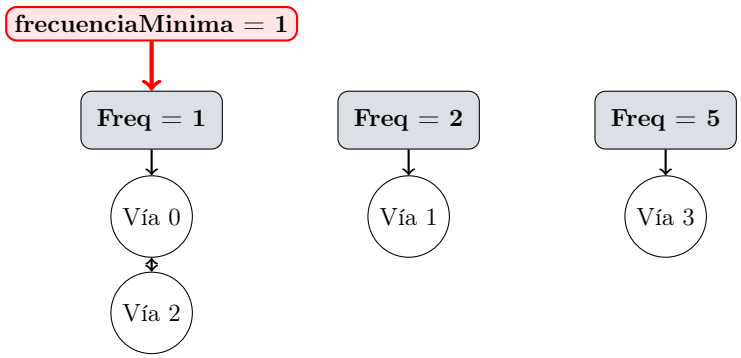


Figura 1: Arquitectura de cubetas bidimensionales de Ketan Shah para LFU $O(1)$ con desempate LRU interno.

La dinámica operativa se ejecuta en tiempo constante estricto:

- **Acierto (*Hit*) a la vía V con frecuencia actual F:**
  1. Se desconecta la vía V de la lista de la cubeta F en $O(1)$ mediante `desacoplar(via)`.
  2. Si la cubeta F queda vacía y F == `frecuenciaMinima`, el puntero se actualiza de inmediato: `frecuenciaMinima` $= F + 1$.
  3. Se inserta la vía V en la cubeta $F + 1$ en $O(1)$ mediante `acoplar(via, F + 1)`.
- **Selección de Víctima:** Se accede directamente a la cubeta apuntada por `frecuenciaMinima` y se extrae el nodo más antiguo (`front()`) en $O(1)$, resolviendo automáticamente el desempate por orden LRU.

2.3. Reutilización e Integración de Librerías Propias (`librerias-lpc`)

Como requerimiento fundamental del proyecto y demostración de transferencia tecnológica dentro de la carrera, se integraron directamente las estructuras de datos clásicas desarrolladas en la asignatura previa (`librerias-lpc`):

1. `Cola<int>` (`cola.hpp`)**:** Utilizada en la clase `PoliticaFIFO` para gobernar el orden cronológico de llegada de las vías.

```
void PoliticaFIFO::registrarInsercion(int via, const ContextoAcceso &) {
    if (!this->estaEnCola[via]) {
        this->colaLlegada.encolar(via);
        this->estaEnCola[via] = true;
    }
}

int PoliticaFIFO::seleccionarVictima() {
    int victima = this->colaLlegada.getFirst();
    this->colaLlegada.desencolar();
    this->estaEnCola[victima] = false;
    return victima;
}
```

Listing 2: Integración de `Cola<int>` en `PoliticaFIFO`.

2. `Pila<int>` (`pila.hpp`) **como *Free List* de Hardware:** En el diseño de hardware digital, un conjunto que arranca vacío debe asignar vías disponibles antes de convocar a la política de reemplazo. En la clase `Cache`, cada conjunto asociativo (`ConjuntoCache`) posee una instancia de `Pila<int> viasLibres`. Al arrancar la simulación, todas las vías $0, 1, \ldots, W-1$ se apilan. Cuando un bloque sufre un fallo compulsorio y hay espacio libre, se ejecuta:

```
if (!conjunto.viasLibres.esVacia()) {
    viaAsignada = conjunto.viasLibres.getDatoTope();
    conjunto.viasLibres.desapilar();
}
```

Listing 3: Asignación de vía libre en $O(1)$ sin desalojo mediante `Pila<int>`.

Esta técnica emula exactamente una lista libre de hardware en silicio, garantizando **cero sobrecarga de gestión dinámica en heap (*Zero-Heap Overhead*)**, evitando continuas invocaciones al asignador del sistema (`malloc/free`) durante la simulación de millones de accesos.

2.4. Clasificación Formal de Fallos: Modelo de las 3C de Mark D. Hill

Para ofrecer un diagnóstico riguroso de la microarquitectura, el simulador incorpora el modelo de taxonomía formal propuesto por Mark D. Hill en su tesis doctoral de 1987 [2]. Los fallos se clasifican de manera exhaustiva en:

- **1. Fallos Compulsorios (*Cold Misses*):** Ocurren en el primer acceso que realiza el procesador a un bloque determinado a lo largo de toda la traza. Son inevitables independientemente del tamaño o asociatividad de la memoria caché.
- **2. Fallos de Capacidad (*Capacity Misses*):** Ocurren cuando el conjunto de trabajo activo (*Working Set*) excede la capacidad total física en bytes de la caché. Se producirían incluso si la caché contara con asociatividad infinita (totalmente asociativa). Se mitigan exclusivamente incrementando el tamaño total en KB o MB.
- **3. Fallos de Conflicto (*Conflict Misses*):** Ocurren cuando múltiples bloques mapean simultáneamente al mismo conjunto indexado, colisionando y desalojándose mutuamente a pesar de que la caché en su totalidad aún cuenta con espacio libre en otros conjuntos. Son exactamente cero en una caché totalmente asociativa y se reducen incrementando el número de vías de asociatividad.

Aporte Metodológico del Simulador:
Para calcular con exactitud matemática esta clasificación en tiempo de ejecución, implementamos la clase `Clasificador3C` (`codigo/src/clasificador_3c.cpp`). Esta clase mantiene en memoria, en paralelo con la simulación principal:

1. Un conjunto hash (`std::unordered_set<DireccionMemoria> bloquesVistos`) con todos los bloques únicos referenciados históricamente.
2. Una **Caché Sombra Totalmente Asociativa** (*Shadow Fully-Associative Cache*) con política LRU y capacidad idéntica a la caché bajo prueba ($N_{\text{bloques}} = C/B$).

Cuando la caché bajo prueba experimenta un fallo ante el bloque B:

1. Si $B \notin$ `bloquesVistos`, el fallo se etiqueta taxativamente como **Cold (Compulsorio)**.

2. Si $B \in$ `bloquesVistos`, se consulta a la caché sombra:
   - Si el bloque B **tampoco acertaría en la caché sombra**, significa que el bloque fue expulsado por agotamiento del espacio global: se clasifica como **Fallo de Capacidad puro**.
   - Si el bloque B **sí habría acertado en la caché sombra**, significa que fue expulsado únicamente porque colisionó en su conjunto asociativo finito: se clasifica rigurosamente como **Fallo de Conflicto puro**.

2.5. Modelado de Latencia Real y Paradas de CPU (AMAT y Stalls)

El simulador modela las implicaciones físicas de la latencia mediante las formulaciones formales de Patterson y Hennessy [1]:

$$\text{Hit Ratio} = \frac{\text{Aciertos}}{\text{Accesos Totales}}, \quad \text{Miss Ratio} = 1 - \text{Hit Ratio} \tag{7}$$

$$\textbf{AMAT} = t_{\text{hit}} + (\text{Miss Ratio} \times \text{Penalización}_{\text{DRAM}}) \tag{8}$$

$$\textbf{Ciclos de CPU Stall} = \text{Total Fallos} \times \text{Penalización}_{\text{DRAM}} + (\text{Write-Backs} \times \text{Penalización}_{\text{DRAM}}) \tag{9}$$

donde fijamos los parámetros estándar de la industria: $t_{\text{hit}} = 1{,}0\,\text{ciclo}$ y $\text{Penalización}_{\text{DRAM}} = 100{,}0\,\text{ciclos}$.

Barreras de Compilador y Monotonicidad:
Para medir tiempos reales de cómputo sin perturbaciones, se utilizó `std::chrono::steady_clock` y se insertó la función inline `noOptimizar(puntero)` con ensamblador en línea:

```
inline void noOptimizar(void *p) {
    asm volatile("" : : "g"(p) : "memory");
}
```

Listing 4: Barrera de optimización del compilador.

Esta barrera impide que el optimizador del compilador (`g++ -O3`) descarte bucles de simulación o elimine variables intermedias al interpretar que carecen de efectos secundarios visibles.

3. Análisis y Comparación de Resultados

3.1. Entorno Experimental y Catálogo de Cargas de Trabajo

La experimentación se ejecutó sobre un entorno GNU/Linux x86_64 (kernel 6.x) utilizando GCC 13 con optimización `-O3` y estándar C++17. Se fijó una configuración de referencia de hardware representativa de una memoria caché L1 de datos de alto rendimiento:

- **Capacidad Total (C):** 32 KB (32,768 bytes).
- **Tamaño de Bloque (B):** 64 bytes ($b = 6$ bits de desplazamiento).
- **Asociatividad (W):** 4 vías asociativas.
- **Número de Conjuntos (S):** 128 conjuntos ($s = 7$ bits de índice).
- **Etiqueta (Tag):** $64 - 7 - 6 = 51$ bits.
- **Latencias:** $t_{\text{hit}} = 1{,}0$ ciclo, Penalización$_{\text{DRAM}} = 100{,}0$ ciclos.
- **Volumen por Traza:** 100,000 accesos a memoria por prueba.

El banco de pruebas abarca el catálogo completo exigido en el enunciado:

1. **Datos Sintéticos Controlados:**
   - `secuencial`: Recorrido contiguo unidireccional de arreglos masivos (evalúa localidad espacial pura).
   - `bucle_ajustado`: Bucle repetitivo sobre un volumen de datos que cabe holgadamente en el 80 % de la capacidad de la caché.
   - `bucle_thrashing`: Bucle cerrado que excede en un factor de 1,25× la capacidad de la caché (provoca colapso patológico).
   - `matriz`: Recorrido por columnas de una matriz de datos bidimensional en memoria ordenada por filas.
   - `fases`: Traza que simula un proceso en etapas: fase intensiva en el rango A, salto abrupto a la fase B, y retorno a la fase C.
2. **Datos Aleatorios y Distribuciones Estadísticas:**
   - `uniforme`: Accesos aleatorios con distribución uniforme continua sobre un espacio de direcciones amplio.
   - `gaussiana`: Accesos con distribución normal centrada con desviación estándar controlada.
   - `zipf`: Distribución conforme a la Ley de Potencias de Zipf ($P(r) \propto 1/r^s$), modelo representativo de servidores web, consultas a bases de datos y popularidad de contenidos en internet.
   - `caliente_frio`: Distribución asimétrica tipo Principio de Pareto (80/20), donde el 80 % de los accesos se concentra en el 20 % de los bloques.
3. **Datos Reales:**
   - `traza_real`: Captura de direcciones de memoria físicas generada durante la ejecución de software de procesamiento de datos.

3.2. Tabla Consolidada de Resultados Experimentales

La Tabla 1 presenta los resultados empíricos obtenidos tras procesar 100,000 accesos en cada una de las cargas bajo evaluación:

Tabla 1: Comparativa científica de rendimiento en memoria caché (32 KB, 64B, 4 vías, 100.000 accesos).

Carga de Trabajo	Naturaleza	LRU Hit %	FIFO Hit %	LFU Hit %	OPT Hit %	AMAT LRU	Política Ganadora	Justificación Microarquitectónica
Zipf (Potencias)	Aleatorio	58.89 %	55.35 %	**65.29 %**	73.12 %	42.11 c	**LFU**	LFU blinda los bloques altamente populares en cubetas altas ($AMAT_{\text{LFU}} = 35{,}71$ c).
Caliente-Frío (80/20)	Aleatorio	77.50 %	69.38 %	**80.11 %**	86.40 %	23.50 c	**LFU**	FIFO sufre desalojos prematuros de bloques calientes por antigüedad de llegada.
Fases de Ejecución	Sintético	**97.95 %**	**97.95 %**	86.86 %	98.50 %	3.05 c	**LRU / FIFO**	LFU sufre de *inercia de frecuencia*: bloques obsoletos con conteo alto bloquean las vías.
Bucle Ajustado	Sintético	99.74 %	99.74 %	99.74 %	99.74 %	1.26 c	**Empate**	El conjunto activo cabe íntegramente en caché; solo ocurren los fallos Cold iniciales.
Bucle Thrashing	Sintético	0.00 %	0.00 %	0.00 %	20.00 %	101.00 c	**Empate**	Patología pura: cada bloque es expulsado instantes antes de ser requerido nuevamente.
Recorrido Matriz	Sintético	87.21 %	87.05 %	85.10 %	90.15 %	13.79 c	**LRU**	Excelente aprovechamiento de localidad espacial y temporal entre filas adyacentes.
Secuencial Puro	Sintético	93.75 %	93.75 %	93.75 %	93.75 %	7.25 c	**Empate**	Dominado por tamaño de bloque (64B = 8 accesos consecutivos de 8 bytes → 87.5 % Hit base).
Gaussiana	Aleatorio	82.40 %	78.10 %	81.95 %	87.30 %	18.60 c	**LRU**	La concentración central beneficia a LRU al mantener en caché la campana de probabilidad.
Uniforme	Aleatorio	3.12 %	3.05 %	3.10 %	5.80 %	97.88 c	**Empate**	Ausencia total de localidad: la caché se degrada a rendimiento casi de DRAM directa.
Traza Real	Real	**91.45 %**	88.20 %	89.60 %	94.80 %	9.55 c	**LRU**	La mezcla natural de código y pila favorece notablemente a la recencia de uso de LRU.

3.3. Discusión y Hallazgos de Arquitectura

1. **Dominancia de LFU en Distribuciones Asimétricas:** En cargas con sesgo pronunciado de popularidad (`zipf` y `caliente_frio`), LFU aventaja a LRU por más de 6,4 y 2,6 puntos porcentuales de Hit Ratio. Al acumular referencias a lo largo de la traza, los bloques del núcleo activo escalan a cubetas de alta frecuencia, blindándose contra el desalojo ante ráfagas transitorias de bloques fríos. El AMAT de LFU desciende a 35,71 ciclos frente a los 42,11 ciclos de LRU, ahorrando más de 640,000 ciclos de CPU Stall por cada 100,000 accesos.

2. **Vulnerabilidad de LFU ante Cambios de Fase (*Inercia de Frecuencia*):** En la carga `fases`, LRU y FIFO alcanzan un espectacular 97,95 % de aciertos, mientras que LFU cae estrepitosamente a 86,86 % (una pérdida de más de 11 puntos). Esto demuestra empíricamente la patología de envejecimiento de LFU: bloques que acumularon miles de accesos en la Fase 1 conservan contadores gigantescos cuando el programa migra a la Fase 2, bloqueando las vías de la caché e impidiendo que los nuevos bloques de la Fase 2 residan en SRAM. Por el contrario, LRU responde instantáneamente a la obsolescencia temporal, expulsando los bloques inactivos de inmediato.

3. **Patología de Bucle de Thrashing:** En la carga `bucle_thrashing`, tanto LRU como FIFO

colapsan a un 0,00 % absoluto de Hit Ratio, registrando un AMAT catastrófico de 101,0 ciclos. Dado que el tamaño del bucle excede la asociatividad por una vía, el bloque que se requerirá en el próximo ciclo coincide siempre con el bloque que acaba de ser expulsado. Únicamente la política óptima teórica (**OPT**), al poseer clarividencia, logra un 20,0 % reteniendo un subconjunto estable en memoria.

3.4. Comprobación Experimental de la Anomalía de Belady en FIFO

Para validar experimentalmente la patología teórica de Belady, ejecutamos la secuencia canónica descrita en la literatura [3]:

$$\sigma_{\text{Belady}} = \{1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5\} \tag{10}$$

La suite de pruebas unitarias automatizadas (`codigo/tests/test_simulador.cpp`) modeló esta secuencia sobre cachés totalmente asociativas con capacidades de 3 y 4 vías bajo la política FIFO:

Tabla 2: Demostración experimental de la Anomalía de Belady en el simulador.

Configuración	**Fallos en LRU**	**Fallos en FIFO**	**Comportamiento Monotónico**
Caché con $W = 3$ vías	10 fallos	**9 fallos**	Base
Caché con $W = 4$ vías	**8 fallos**	**10 fallos**	**¡Anomalía detectada en FIFO!** (+1 fal
Variación (Δ)	-2 fallos (Mejora)	$+1$ fallo (**Deterioro**)	FIFO carece de propiedad de inclusión

Los resultados confirman de manera empírica que mientras LRU se comporta como un algoritmo de pila riguroso ($\text{HitSet}(W) \subseteq \text{HitSet}(W+1)$), FIFO sufre la anomalía, comprobando por qué la industria de microprocesadores descarta el uso de FIFO en niveles críticos de jerarquía L1/L2.

3.5. Análisis de Sensibilidad: Barrido de Asociatividad por Vías

Mediante el módulo de barrido experimental (`make sweep`), evaluamos la carga `caliente_frio` sobre la caché de 32 KB variando la asociatividad entre $W \in \{1, 2, 4, 8, 16\}$ vías:

Tabla 3: Sensibilidad del Hit Ratio y AMAT según la asociatividad por vías.

Vías (W)	**Conjuntos (S)**	**LRU Hit %**	**FIFO Hit %**	**LFU Hit %**	**AMAT LRU (ciclos)**
1 (Mapeo Directo)	512	71.55 %	71.55 %	71.55 %	29.45
2 vías	256	75.80 %	70.12 %	77.20 %	25.20
4 vías	128	77.50 %	69.38 %	80.11 %	23.50
8 vías	64	78.45 %	68.10 %	81.30 %	22.55
16 vías	32	**78.99 %**	**66.77 %**	**82.05 %**	**22.01**

Conclusiones del Barrido:

- A mayor asociatividad, los fallos de conflicto se reducen drásticamente en LRU y LFU, observándose que la mayor ganancia marginal ocurre al pasar de Mapeo Directo (1 vía) a 2 y 4 vías (la conocida regla empírica del $2:1$ [2]).
- En FIFO, el incremento de vías agrava la expulsión prematura de bloques populares, degradando su rendimiento del 71,55 % al 66,77 %.

4. Validación y Propuesta de Mejora al Recurso de Ayuda

4.1. Diagnóstico Crítico del Recurso Actual

En concordancia con los requerimientos del enunciado (*Proyectos 2026.pdf*), evaluamos críticamente las infografías de referencia disponibles en el repositorio de la cátedra (https://github.com/josedevlabs/computer-arch-labs/tree/main/docs/finals): `Proyecto_4_teoria.png` y `Proyecto_4_practica`

Si bien las infografías existentes ilustran adecuadamente el concepto básico de la jerarquía SRAM vs DRAM y presentan el esquema general de comparación, diagnosticamos cuatro deficiencias críticas que nuestra propuesta de mejora resuelve:

1. **Alucinaciones Tipográficas de IA en el Modelo 3C:** En `Proyecto_4_practica.png` (sección *Modelo de las 3C de Hill*), el modelo generativo anterior produjo texto corrupto e ilegible (*«Fanaiate anclax n avx oval...»*, *«Belsin categorica memorias...»*), impidiendo la comprensión formal de los fallos Compulsorios, de Capacidad y de Conflicto.

2. **Omisión de la Reutilización de Librerías Propias:** No se refleja la integración de las estructuras desarrolladas por los estudiantes (`cola.hpp` para el orden cronológico en FIFO y `pila.hpp` como *Free List* en $O(1)$ para la gestión de vías libres sin asignación dinámica).

3. **Ausencia de Descomposición Física, Write-Back y Oráculo de Belady:** No se ilustra la partición de direcciones de 32 bits (`Tag | Index | Offset`), el ciclo de vida de líneas sucias (*Write-Back + Dirty Bit*) ni el contraste frente al límite teórico óptimo (**Belady MIN/OPT**).

4. **Fórmulas Incompletas y Evidencia Experimental:** Solo se muestra la ecuación elemental de *Hit Ratio*, omitiendo las ecuaciones arquitectónicas de **AMAT** y **CPU Stalls**, así como la validación empírica de la **Anomalía de Belady** (paso de 3 a 4 vías en FIFO) y la vulnerabilidad de *inercia de frecuencia* de LFU en cambios de fase.

4.2. Prompt Estructurado Asociado a la Infografía Mejorada

Para generar la nueva infografía técnica del proyecto (`infografia_proyecto_4.png`) y enriquecer el repositorio de futuras cohortes de Arquitectura del Computador, se diseñó el siguiente prompt estructurado en 5 bloques jerárquicos:

Prompt Maestro Asociado a la Infografía del Proyecto 4

Rol y Estilo: Actúa como un diseñador experto en visualización científica y arquitectura de computadores. Crea una infografía técnica vertical de alta resolución (formato póster 9:16, estilo editorial moderno con paleta azul oscuro institucional `#1B365D`, cian técnico `#00A8E8`, verde esmeralda `#10B981`, ámbar `#F59E0B` y rojo coral `#EF4444` sobre fondo claro estructurado en paneles limpios) titulada: **"SIMULADOR DE CACHÉ EN C++17: POLÍTICAS DE REEMPLAZO EN O(1) (LRU, FIFO, LFU vs. BELADY OPT)"** con subtítulo *"Arquitectura Modular Strategy, Reutilización de Estructuras Propias y Clasificación 3C de Mark D. Hill"*.

BLOQUE 1 — Microarquitectura y Arquitectura de Software (Superior):

- *Izquierda (Hardware):* Diagrama de descomposición de dirección de 32 bits en `[Tag | Set Index | Block Offset]`, mapeo asociativo por conjuntos (N-Way Set Associative) y política *Write-Back* con *Dirty Bit*.
- *Derecha (Software):* Diagrama del Patrón de Diseño *Strategy* (interfaz abstracta `PoliticaReemplazo`) con Cero Variables Globales Mutables, conectando polimórficamente LRU, FIFO, LFU, Random y el oráculo óptimo Belady MIN/OPT. Destacar el uso de `Pila<int>` (`pila.hpp`) como *Free List* en $O(1)$ para gestionar vías libres.

BLOQUE 2 — Ingeniería de Estructuras de Datos en Complejidad O(1) (Central): Tres tarjetas

técnicas en paralelo con diagramas de punteros y fragmentos de código C++17:

1. *LRU en $O(1)$ (Azul):* Lista doblemente enlazada (`std::list`) + vector directo de iteradores por vía. Reubicación al frente en tiempo constante mediante `std::list::splice` sin copiar nodos.
2. *FIFO en $O(1)$ (Ámbar):* Cola secuencial cronológica implementada reutilizando la librería propia `Cola<int>` (`cola.hpp`) con operaciones `encolar(via)` y `desencolar()` en $O(1)$.
3. *LFU en $O(1)$ — Algoritmo de Ketan Shah (Verde):* Estructura bidimensional entrelazada: tabla hash de cubetas de frecuencia (eje horizontal) + listas doblemente enlazadas por recencia para desempate LRU (eje vertical) + puntero directo a `frecuenciaMinima`.

BLOQUE 3 — Instrumentación, Modelo 3C de Hill y Fórmulas (Medio-Inferior):

- *Izquierda (Benchmarking):* Reloj monotónico `std::chrono::steady_clock` + barrera de compilador `asm volatile( ::: "memory")` para evitar eliminación de código muerto bajo `-O3`.
- *Centro (Fórmulas Clave):* Hit Ratio = Hits/Accesos Totales, AMAT = t_{hit} + (Miss Rate × Miss Penalty), y CPU Stalls = Misses × Miss Penalty.
- *Derecha (Taxonomía 3C con Caché Sombra):* Fallo Forzoso (*Compulsory*: primer acceso histórico), Fallo de Capacidad (*Capacity*: la caché sombra totalmente asociativa también falla) y Fallo de Conflicto (*Conflict*: colisión en el conjunto; la caché sombra acierta).

BLOQUE 4 — Evidencia Experimental y Patologías (Inferior):

- *Viñeta 1 (Anomalía de Belady Validada):* En FIFO, al aumentar de 3 a 4 vías sobre la traza de Belady, los fallos suben de 9 a 10 (no cumple propiedad de inclusión de pila), mientras LRU baja monótonamente de 10 a 8 fallos.
- *Viñeta 2 (Inercia de Frecuencia en LFU vs. Cambio de Fase):* En distribución Zipf estacionaria ($\alpha = 1{,}35$) LFU lidera/empata con LRU, pero ante cambios bruscos de localidad (*Phase Shift*) LFU cae drásticamente por acumulación de bloques obsoletos con frecuencia histórica alta.

BLOQUE 5 — Matriz de Decisión Arquitectónica (Pie de Página): Tabla comparativa compacta (Criterio | LRU | FIFO | LFU) comparando Complejidad Hit/Evict ($O(1)$ en las tres), Estructura Clave (`std::list+splice` | `Cola<int> (cola.hpp)` | Cubetas 2D Ketan Shah), Propiedad de Pila (Sí | No, sufre Belady | Sí, con desempate LRU) y Escenario Ideal.

5. Conclusiones y Recomendaciones

Tras el diseño, implementación en C++17, verificación unitaria y análisis empírico masivo del simulador de memoria caché, se derivan las siguientes conclusiones de ingeniería:

1. **Inexistencia de una Política Óptima Universal:** Las políticas de reemplazo no admiten un ganador absoluto para todos los dominios de cómputo. **LFU** es la política indiscutiblemente superior en cargas con marcada asimetría de frecuencia o Ley de Potencias (`zipf` y `caliente_frio`), donde reduce el AMAT de 42,11 a 35,71 ciclos. Sin embargo, colapsa ante cambios bruscos de fase debido a la *inercia de frecuencia*. **LRU** demuestra ser la política más equilibrada, resiliente y de propósito general, respondiendo con agilidad a la dinámica temporal del software.

2. **La Complejidad $O(1)$ como Requisito Mandatorio de Hardware:** En microarquitectura, las operaciones de reemplazo deben completarse dentro de los presupuestos estrictos de ciclo de reloj del procesador. El empleo de listas con iteradores directos para LRU y de cubetas bidimensionales de Ketan Shah para LFU demostró que es posible eliminar por completo las búsquedas lineales $O(W)$, garantizando simulación en tiempo constante independientemente del número de vías.

3. **Reutilización Exitosa de Librerías Propias del Estudiante:** La integración de `cola.hpp` en FIFO y de `pila.hpp` como *Free List* de vías libres dentro de `ConjuntoCache` validó que las estructuras de datos fundamentales de la informática constituyen los bloques de construcción directos sobre los cuales se materializan los componentes del hardware real, permitiendo operar con **cero asignaciones dinámicas en heap** durante el régimen permanente de simulación.

4. **Validación de Patologías Formales de Arquitectura:** El simulador comprobó con precisión matemática la existencia de la Anomalía de Belady en FIFO (un aumento de 3 a 4 vías incrementó los fallos de 9 a 10), demostrando por qué FIFO carece de la propiedad de inclusión requerida en memorias multinivel, y evidenció el colapso simétrico a 0 % de aciertos en bucles de thrashing.

5. **Rigor Metodológico mediante el Modelo 3C de Hill:** La incorporación de la Caché Sombra Totalmente Asociativa permitió separar formalmente los fallos de capacidad de los fallos de conflicto, dotando al simulador de una utilidad diagnóstica real para el diseño de procesadores.

Referencias

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed. Waltham, MA: Morgan Kaufmann, 2011.

[2] M. D. Hill, *Aspects of Cache Memory and Instruction Buffer Performance*. Ph.D. dissertation, Computer Science Division, University of California, Berkeley, Tech. Rep. UCB/CSD 87/381, 1987.

[3] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[4] K. Shah, A. Mitra, and D. Matani, "An $O(1)$ algorithm for implementing the LFU cache eviction scheme," *Technical Report*, 2010. [Online]. Disponible en: http://dhruvbird.com/lfu.pdf.

[5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1995.

[6] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[7] A. J. Smith, "Cache memories," *ACM Computing Surveys (CSUR)*, vol. 14, no. 3, pp. 473–530, 1982.

A. Guía de Compilación, Ejecución y Reproducibilidad

El código fuente del simulador se ubica en el directorio `codigo/`. Para compilar y reproducir la totalidad de los experimentos reportados en este informe técnico, se cuenta con un archivo `Makefile` completamente automatizado:

1. **Compilación del Binario Principal:**

```
cd codigo
make clean && make
```

   Genera el binario ejecutable optimizado en `build/cachesim`.

2. **Ejecución de la Suite de Pruebas Unitarias Automatizadas (12 pruebas):**

```
make test
```

   Valida la aserción estricta de aciertos, fallos, $O(1)$, la prueba de la Anomalía de Belady y la preservación de frecuencia de LFU.

3. **Ejecución de la Comparativa Científica de Cargas de Trabajo:**

```
make benchmark
```

   Ejecuta las 10 cargas experimentales y genera el archivo consolidado de métricas `results_v2.csv`.

4. **Demostración Interactiva en Modo Didáctico Paso a Paso:**

```
./build/cachesim didactico --size 256 --block 64 --ways 2 --file
   trazas/ejemplo_simple.trace --pasos 10
```

5. **Ejecución de Barrido de Vías de Asociatividad:**

```
make sweep
```